

Spend Bytes on Breadth: Precision–Count Trade-offs for Decode-Time KV Compression in Long Chain-of-Thought Reasoning

Runguo Li

University of Illinois Urbana-Champaign
runguol2@illinois.edu

Abstract

Reasoning models write most of their KV cache while decoding long chains of thought (CoT), so the cache has to be compressed online under a fixed memory budget. Decode-time methods mostly decide *which* tokens to evict. We ask how a fixed byte budget should be split between the number of cached tokens and their precision. BreadthKV spends the bytes on more tokens at low precision, combining quantization with eviction, and picks the bit-width for each model and budget with a 60-problem end-to-end calibration, since offline attention error does not predict it reliably. On three reasoning models and four math and science benchmarks, it scores above eviction alone in 17 of 18 settings and produces shorter outputs. Much of what eviction loses comes from derailed runs, which keep reasoning until the length cap without reaching an answer. On Qwen3-8B at our tightest budget, eviction sends 91% of AIME samples to the cap and BreadthKV 40%. Under the same protocol, BreadthKV is statistically indistinguishable from a joint rate–distortion allocator (RDKV) that uses 27% more KV memory-time, and it outperforms our re-implementation of ThinKV.

1 Introduction

Long-CoT reasoning models generate 10–30K tokens per problem, and the KV cache grows with every decoded token. At 32K tokens, a single Qwen3-8B sequence holds 4.5 GiB of KV cache in bf16, so four sequences already take more memory than the model’s 16 GB of weights. The cache has to be compressed online, and there is no fixed query to guide it, as there is when compressing a prompt. Token importance also shifts during decoding, since reflection steps often return to early intermediate results, and errors accumulate over the trace. Under eviction, models can fall into loops that run until the length limit (Chang et al., 2026b; Cheong et al., 2026), and a smaller cache can make the

model write more tokens, which offsets part of the memory saved (Liu et al., 2025; Zhu et al., 2026).

Recent decode-time methods decide *which* tokens to keep, using attention scores (Zhang et al., 2023; Li et al., 2024), redundancy-aware scores (Cai et al., 2025), thought-type heuristics (Ramachandran et al., 2026), or value ranges combined with stochastic eviction (Chang et al., 2026b). Concurrent work finds that uniform random eviction, with the prompt kept, does about as well as these scores at $4\times$ compression (Wang et al., 2026b).

A simple offline measurement shows why the other axis, precision, deserves attention. Even an oracle that knows which tokens future queries will attend to loses about 13% of their attention mass when it keeps 10% of the tokens (§4). A long CoT keeps going back to its own intermediate results, and an evicted token cannot be recovered. A token stored at 2 bits is noisy, but a later reflection step can still find it. At a fixed byte budget, a head can keep K tokens in fp16, about $3K$ at 4 bits or about $6K$ at 2 bits, so the question becomes how to trade noise per token against coverage. Zhang et al. (2025) studied this trade-off for long inputs, where pruning plus quantization beats pruning at equal memory. Long-CoT decoding differs in two ways: the cache is written during generation, so errors compound over tens of thousands of steps, and a lossy cache changes what the model writes, including how long it runs. Kim et al. (2026) compare quantization and eviction for reasoning models, but separately, and find 2-bit quantization lossy.

We combine the two at equal KV bytes and much tighter budgets. On three models and four benchmarks, the byte split matters more than the token scorer. With the bit-width chosen by a short end-to-end calibration, a method we call BreadthKV, the low-bit hybrid scores above SnapKV-style eviction in 17 of 18 settings (§5.1). The best bit-width depends on the model and budget, and neither offline

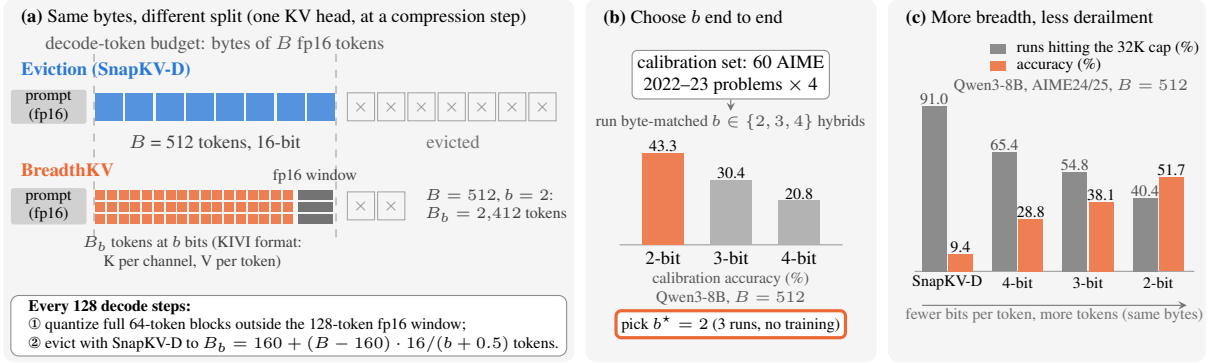


Figure 1: Overview of BreadthKV. (a) At the same KV byte budget per head, eviction keeps B full-precision decode tokens, whereas BreadthKV quantizes older tokens to b bits in the KIVI format and keeps B_b of them plus a 128-token fp16 window; both compress every 128 decode steps and keep the prompt in full. (b) The bit-width is chosen for each model and budget by running the byte-matched $\{2, 3, 4\}$ -bit hybrids on 60 held-out AIME problems. (c) At equal bytes, fewer bits per token lower the share of runs that reach the 32K cap and raise accuracy (FullKV: 10.2% capped, 69.8% accuracy). Squares are not to scale.

attention error nor a per-head online probe finds it (§5.2–5.3). Eviction loses mostly through derailed runs that never conclude, and keeping more tokens at lower precision makes them rarer, down to about 2 bits (§5.6–5.8). We also run RDKV (Zhang et al., 2026b) with the authors’ code, where we found and fixed a zero-point clamp in the released quantizer, and our re-implementation of ThinkKV (Ramachandran et al., 2026) under the same protocol (§5.4–5.5). Last, we measure the latency of an int4 decode kernel (§5.9).

2 Related work

Which tokens to keep. Most KV compression work selects tokens, and for long inputs it does so by position, accumulated attention or value-aware scores (Xiao et al., 2024; Zhang et al., 2023; Li et al., 2024; Cai et al., 2024; Yang et al., 2024a; Sengupta et al., 2025), and Ada-KV (Feng et al., 2025) splits the budget across heads. Reasoning models need eviction during decoding, and a fast-growing line of methods targets that case (Cai et al., 2025; Zhang et al., 2026a; Hu et al., 2025; Liao et al., 2025; Mao et al., 2026; Kolawole and Smith, 2026; Zeng et al., 2026; Liu et al., 2026b; Du et al., 2026). Two papers are most relevant here. Random Attention (Wang et al., 2026b) shows that random eviction with the prompt kept matches the scored methods at about $4\times$ compression, which suggests that better scores are not where the remaining accuracy lies. VaSE (Chang et al., 2026b) finds that evicting a few value states with large ranges makes generations loop, restating intermediate reasoning without progress, and names a high-precision re-

serve for such values as future work, which hints that precision and looping are linked. All of these methods keep the selected tokens at full precision. We fix the scorer (SnapKV-D) and vary the precision. SeerAttention-R (Gao et al., 2025) learns sparse attention for long decoding but keeps the full cache, so it saves compute and not memory.

How many bits. KIVI (Liu et al., 2024) and KVQuant (Hooper et al., 2024) quantize keys per channel and values per token, the format we use. Later work allocates bits more finely, by layer (Li et al., 2025), channel (Xia et al., 2026), token (Boroujeni et al., 2026; Shen et al., 2026), query-aware channel selection for reasoning (Zhang et al., 2026c), or over time, as PM-KVQ (Liu et al., 2026a) lowers precision while a long CoT grows, and other work improves the quantizer itself (Muller et al., 2026; Zuo et al., 2026; Chauhan et al., 2026; Ashkboos et al., 2024; Zandieh et al., 2025). Without eviction, even a 2-bit cache of a 32K-token trace does not fit our budgets, so these methods are complementary to our question. BreadthKV uses one bit-width per model and budget, and §5.4 tests whether finer allocation helps.

Trading precision for count (closest work). *More Tokens, Lower Precision* (Zhang et al., 2025) shows that at equal memory, pruning combined with quantization, for example $4\times$ the tokens at 4 bits, outperforms pruning alone. The study covers long-input prefill (LongBench, NIAH, RULER) on non-reasoning models, where 2-bit precision degrades sharply and about 4 bits is recommended. We study decode-time compression of long CoT,

in which the cache is written during generation and a lossy cache changes how long the model runs. At our budgets, calibration picks 2 or 3 bits in 5 of 6 model–budget pairs, and always using 4 bits is significantly worse (§5.3). *Not All Bits Are Equal* (Kim et al., 2026) compares eviction (R-KV, StreamingLLM) and quantization (HQQ, per channel) separately on Qwen3 models from 0.6B to 32B. It counts weight memory in the budget and uses generation budgets of 2K–30K tokens and eviction budgets of 2K–8K tokens. It prefers eviction for models below the effective size of an 8-bit 4B model and finds 2-bit quantization lossy. Our 7–8B bf16 models are above that threshold, we combine quantization and eviction at equal KV bytes, and our eviction budgets (512–1024 tokens) are $2\text{--}16\times$ smaller. Joint allocators decide per token or element between several precisions and eviction. MiKV (Yang et al., 2024b) keeps evicted tokens at low precision, HqeKV (Wang et al., 2026a) lets an optimizer choose the action for each element, QEvict (Garg et al., 2026) keeps full-precision, INT2 and evicted tiers at matched measured memory on long inputs, ThinKV (Ramachandran et al., 2026) sets precision and retention by thought type, and RDKV (Zhang et al., 2026b) solves a joint rate–distortion allocation. ThinKV reports that hybrids trace a Pareto frontier and partly offset the length inflation of pure quantization, which reaches $5.1\times$. These works propose allocators, whereas we vary only the byte split, with the scorer and format fixed, and evaluate ThinKV and RDKV under the same protocol.

Length inflation and loops. Compression lengthens reasoning, both for KV eviction, as Hold Onto That Thought (Liu et al., 2025) and ReCo (Zhu et al., 2026) report, and for weight quantization (Lian et al., 2026), and Pipis et al. (2026) study why reasoning models loop in the first place. KV-Rescue (Cheong et al., 2026) separates incoherent generation from repetition loops under eviction and repairs runs with reasoning steps from a small full-context model, and SkipKV (Tian et al., 2025) suppresses redundant sentences. Ai et al. (2026) show that compression can keep final answers intact while degrading the reasoning behind them. Benchmarks of KV compression cover long inputs (Yuan et al., 2024) and reasoning (Liu et al., 2026c, 2025). We add a controlled measurement of how the share of runs that reach the cap moves with the byte split (§5.6) and test whether forcing a conclusion helps (§5.8).

Low-rank projection. Palu (Chang et al., 2025), xKV (Chang et al., 2026a), STAR-KV (Bhatnagar et al., 2026) and ShadowKV (Sun et al., 2025) compress the hidden dimension. In our pilot, decode-time values are far from low-rank and their subspace drifts along the trace (§4), so we did not pursue this direction.

Random Attention, QEvict, ReCo, KV-Rescue and Ai et al. (2026) appeared within three months of this work and are concurrent with it.

3 Byte-budgeted precision–count compression

Budget. For each sequence, layer and KV head, the keys and values of decode tokens are limited to the bytes of B fp16 tokens. The prompt is kept in full and not counted, for every method. All methods compress every 128 decode steps and bring each head back to its budget.

Eviction (SnapKV-D). Decode-time SnapKV keeps each head’s top- B decode tokens by the attention they receive from the last 8 queries, max-pooled over 7 positions, and always keeps the 8 most recent tokens. R-KV, H2O, StreamingLLM and random eviction follow the same schedule with their own scores.

Precision-for-count hybrid (b -bit + SnapKV-D). The hybrid uses the same eviction, but decode tokens older than a 128-token fp16 residual window are quantized in complete 64-token blocks in the KIVI format (Liu et al., 2024). K is quantized per channel within each 64-token group and V per token within 64-channel groups. Both use asymmetric quantization with an fp16 scale and zero point per group, which costs $b + 0.5$ bits per element. To match bytes, we raise the token budget to

$$B_b = 160 + (B - 160) \cdot \frac{16}{b + 0.5}, \quad (1)$$

where 160 approximates the fp16 tail (128–191 tokens), so hybrids stay within about 5% of the budget; memory-time is measured exactly. Rounded down, this gives 1411/1769/2412 tokens at 4/3/2 bits at $B = 512$, and 3232/4109/5689 at $B = 1024$.

BreadthKV: choosing b by end-to-end calibration. We run the byte-matched $\{2, 3, 4\}$ -bit hybrids on a held-out calibration set (60 AIME 2022–23 problems $\times$ 4 samples) and keep the most accurate b for each model and budget, breaking ties toward fewer bits. Calibration takes three runs per model

Table 1: Relative attention-output error at equal bytes (R1-Distill-Qwen-7B, 8 MATH500 traces). Window: SnapKV-style window attention.

bytes	eviction			<i>b</i> -bit + eviction		
	random	window	oracle	low-rank	2-bit	4-bit
5%	0.403	0.244	0.191	0.314	0.206	0.162
10%	0.365	0.200	0.144	0.284	0.175	0.104
20%	0.298	0.146	0.093	0.249	—	0.046

and budget, needs no training and uses no test problems. We call the byte-matched hybrid with the calibrated *b* BreadthKV and use it as the reference in all comparisons.

4 Pilot analyses

Before running anything end to end, we checked offline whether breadth can help at all. We take teacher-forced FullKV traces of R1-Distill-Qwen-7B on 8 MATH500 problems and cut each trace at 30%, 60% and 90% of its length. At each cut, the KV of decode tokens older than the recent window is compressed to a fraction *r* of its fp16 bytes, and we measure the relative error of the attention output of the next 256 queries over all layers (Table 1). Quantizing to 4 bits and evicting with a simple window score gives lower error than oracle eviction at every byte ratio. Eviction also has a floor that no scorer can remove. Even with exact knowledge of future attention, it loses about 13% of the future attention mass at 10% retention. Low-rank projection fits these caches poorly, since V needs 78 of its 128 dimensions to keep 90% of its energy, and its top-16 subspace drifts between the first and last thirds of a trace (overlap 0.69).

5 Experiments

Setup. We evaluate R1-Distill-Qwen-7B, R1-Distill-Llama-8B (Guo et al., 2025) and Qwen3-8B in thinking mode (Yang et al., 2025). We report pass@1 on AIME24 and AIME25 with 8 samples per problem, on MATH500 with one sample, and on GPQA-Diamond with 4 samples (R1-Qwen only). We decode with temperature 0.6 and top-*p* 0.95 (top-*k* 20 for Qwen3), up to 32K tokens (16K for MATH500), at budgets $B \in \{512, 1024\}$. AIME24 with R1-Qwen served as our dev set while we developed the methods (Appendix B), so we exclude it from the test results. For statistics, we use a paired bootstrap over problems (5000 resamples, 95% CI); pooled tests over settings resample prob-

Table 2: Accuracy gain (pp) of BreadthKV over SnapKV-D at the same budget, $B = 512 / 1024$. *95% CI includes 0. †R1-Qwen on AIME24 is our dev set and is not reported; all other cells are test results.

benchmark	R1-Qwen-7B	R1-Llama-8B	Qwen3-8B
AIME24	dev [†]	+20.8 / +8.3	+49.2 / +50.0
AIME25	+12.9 / +10.0	+6.7* / +5.8	+35.4 / +51.2
MATH500	+8.6 / +5.8	+6.8 / −0.8*	+17.4 / +11.6
GPQA	+7.3 / +6.8	—	—

lems within each setting. We use our own decoding engine on HF weights, with FlashAttention-2 decoding (Dao, 2024), CUDA graphs and per-head variable-length caches. The engine matches HF (teacher-forced top-1 agreement $\geq 98\%$) and reproduces the reported FullKV accuracy of R1-Qwen (MATH500 93.0 vs. 92.8; AIME24 55.4 vs. 55.5). Appendix A gives prompts, grading, data sources, software and compute, and Appendix C lists the results for every setting.

5.1 Main results

Keeping more tokens at low precision beats keeping fewer tokens in fp16, and on Qwen3-8B the difference is large. Table 2 gives BreadthKV’s gain over SnapKV-D at the same budget. The gain is positive in 17 of 18 settings, its CI excludes zero in 16, and pooled over all settings it is 17.4 pp [15.6, 19.3]. The one negative case is R1-Llama MATH500 at $B = 1024$, where SnapKV-D is already within 2.4 pp of FullKV. Choosing the bit-width per setting on the test set, an oracle, would add only 0.83 pp on average (§5.3). Pure eviction nearly collapses on Qwen3-8B, which scores 5.8% on AIME25 at $B = 512$ with SnapKV-D and 65.4% with FullKV, while BreadthKV at $B = 1024$ comes close to FullKV (Table 9 in Appendix C).

The choice of eviction scorer matters much less. BreadthKV is above the best of five eviction methods (SnapKV-D, R-KV, H2O, StreamingLLM and random) in 17 of 18 settings. Among them, random eviction matches SnapKV-D on the R1-Distill models and does better on Qwen3-8B, in line with Wang et al. (2026b), and H2O is the weakest (we ran StreamingLLM and H2O at $B = 1024$ only). A rank-32 low-rank baseline with a frozen basis collapses. Without eviction, KIVI-4 stays within 5.5 pp of FullKV, but it holds $0.29\text{--}0.34\times$ FullKV’s memory-time, against $0.08\text{--}0.22\times$ for the byte-matched hybrids.

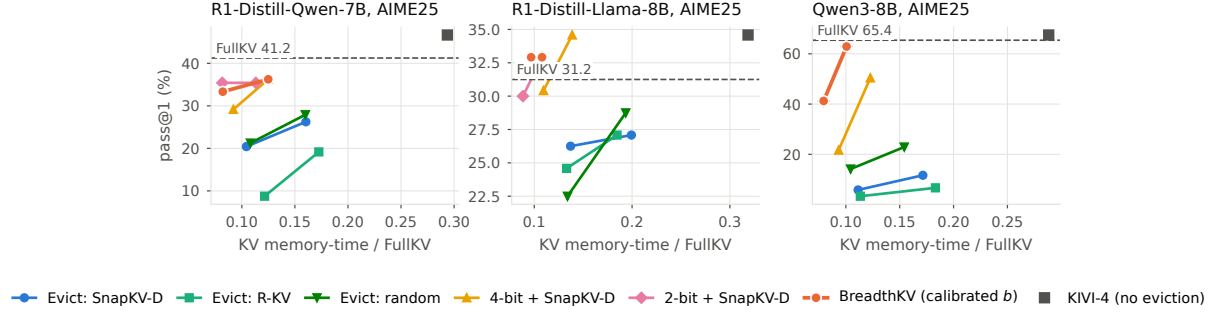


Figure 2: Accuracy vs. KV memory-time (relative to FullKV) on AIME25. Each line connects a method’s results at $B = 512$ and 1024 ; upper left is better. BreadthKV uses the calibrated bit-width, so its points coincide with the hybrid of that width where one is plotted (on Qwen3-8B it covers the 2-bit line). The square is KIVI-4 without eviction, and the dashed line is FullKV.

Length and memory-time. SnapKV-D outputs are $1.1\text{--}2.0\times$ as long as FullKV’s, and BreadthKV’s are $0.76\times$ as long as SnapKV-D’s (pooled). The memory a cache occupies is the bytes it holds over time, so longer outputs offset part of the savings that retention ratios suggest. On Qwen3 AIME25 at $B = 1024$, SnapKV-D holds $0.17\times$ FullKV’s memory-time and scores 11.7%, while BreadthKV holds $0.10\times$ and scores 62.9% (Fig. 2).

5.2 The best bit-width depends on budget and model

No single bit-width is best everywhere. On our dev set, the best width shared by all heads rises with the budget, from 2 bits at $B = 256$ to 8 bits at $B = 2048$ (Appendix B). On the test sets the preference depends on the model. For the R1-Distill models, the 2-, 3- and 4-bit hybrids are close at $B = 1024$, while Qwen3-8B clearly prefers 2 bits (Table 9). Qwen3-8B also writes the longest AIME traces (15.5–18.5K tokens on average, against 13–14K for the R1-Distill models). This fits the idea that longer traces need more coverage, but with three models we cannot separate trace length from other differences between them.

Per-head online allocation did not help. Our first approach chose the width online for each head (PACT, Appendix B). Offline, letting each head pick its own width lowers attention error by 20–42%, and a cheap probe on recent queries recovers most of that gain, but end to end the per-head version scored below a fixed 4-bit width on the dev set and was indistinguishable from fixed widths on the test sets (1 of 18 comparisons significant). A width table tuned on the dev set did not transfer either; on Qwen3-8B it trails the 2-bit hybrid in all

six AIME and MATH500 settings. More generally, attention error at a single step predicts accuracy poorly. Random eviction, for example, has $1.8\times$ the attention error of window-based eviction and the same dev accuracy.

5.3 Choosing the bit-width by end-to-end calibration

Since neither offline error nor online probes find the best width, we measure it directly. Offline distortion curves at equal bytes, computed on 6 FullKV AIME24 traces per model (not used to choose b), get the preference wrong on two of the three models; for Qwen3-8B they favor 4 over 2 bits, the opposite of what happens end to end. Three calibration runs per model and budget pick the widths in Table 3, and Table 4 and Fig. 3 show how the choice does on the 18 held-out settings.

Calibration beats always using 4 bits by 4.5 pp pooled [+3.2, +5.8] and always using 3 bits by 2.1 pp [+1.2, +3.1]. It is not significantly better than always using 2 bits (+0.4 pp [−0.2, +1.0]). Calibration matters because each fixed width does badly on some model, 2 bits on R1-Llama (mean regret 3.1 pp) and 4 bits on Qwen3 (12.7 pp), while calibration stays within 1.3 pp on every model. On Qwen3, where a wrong choice hurts most, the choice is also stable. Resampling the calibration problems picks 2 bits in 100% and 99% of resamples at the two budgets.

5.4 Comparison with a joint rate–distortion allocator (RDKV)

RDKV (Zhang et al., 2026b) is the prior method closest to our view of trading precision for count, so we reproduce it with the authors’ code. It treats eviction and quantization as one rate–distortion

Table 3: Calibration accuracy (%) of 2/3/4-bit hybrids and the chosen b .

model	@512	@1024
R1-Qwen-7B	28.3/ 31.2 /28.3 $\rightarrow$ 3	33.8/35.0/ 40.0 $\rightarrow$ 4
R1-Llama-8B	33.8/ 36.7 /35.8 $\rightarrow$ 3	40.0/40.0 /39.2 $\rightarrow$ 2
Qwen3-8B	43.3 /30.4/20.8 $\rightarrow$ 2	62.9 /57.1/50.0 $\rightarrow$ 2

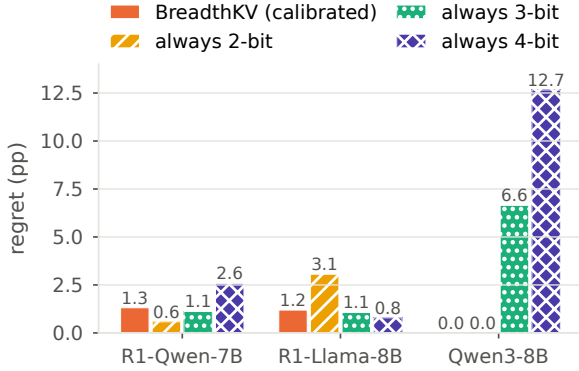


Figure 3: Held-out regret of bit-width selection strategies: the mean gap (pp) to the best bit-width over each model’s six test settings (two budgets $\times$ three benchmarks); lower is better.

problem. For each KV head, a Lagrangian knapsack assigns each V token and each K channel a width in $\{0, 2, 4, 8, 16\}$ bits, where 0 means the token is evicted or the channel is pruned, and weights a calibrated distortion table $\epsilon(b)$ by attention-based importance.

Reproduction. The released code compresses the prompt once after prefill and provides a block-wise mode for decoding, which we use. Decode tokens are grouped into 128-token blocks. When the next block is complete, a block is scored with the queries of that next block, allocated once by the same knapsack under a per-block budget of b fp16-token equivalents, and then frozen. We call the upstream scoring, knapsack solver and ϵ calibration directly, and run the calibration on traces from our calibration set. A unit test checks that our batched allocation matches upstream `_perhead_select_and_pack` bit for bit. Because the block mode has no total budget, we choose b for each model and budget so that RDKV’s mean KV memory-time on the calibration set matches that of BreadthKV. This gives $b = 4$ at $B = 512$ and $b = 13/15/11$ at $B = 1024$ for R1-Qwen/R1-Llama/Qwen3. As for all methods, the prompt is kept in full.

The released quantizer clamps the zero point.

Table 4: Held-out regret relative to the best bit-width in each setting (18 settings).

strategy	mean	worst	>3 pp
BreadthKV (calibrated)	0.83	3.4	1/18
always 2-bit	1.21	7.5	2/18
always 3-bit	2.94	14.6	4/18
always 4-bit	5.34	26.2	7/18

Table 5: BreadthKV (BKV) vs RDKV (block-wise, fixed quantizer): accuracy (%) averaged over each model’s three test benchmarks (R1-Qwen: AIME25, MATH500, GPQA; others: AIME24, AIME25, MATH500), and RDKV’s KV memory-time relative to BreadthKV (MT).

model	B	BKV	2-bit	RDKV	MT
R1-Qwen	512	51.8	53.8	49.3	1.20 $\times$
R1-Qwen	1024	55.0	54.5	55.2	1.14 $\times$
R1-Llama	512	58.3	54.6	54.5	1.18 $\times$
R1-Llama	1024	56.4	56.4	57.9	1.44 $\times$
Qwen3	512	64.6	64.6	65.2	1.30 $\times$
Qwen3	1024	77.1	77.1	75.1	1.37 $\times$

The released `quantize_and_pack` sets the zero point to `clamp(round($-\min/s$), 0, $2^b - 1$), which clips groups whose range does not contain 0. Post-RoPE low-frequency K channels often keep the same sign throughout a 128-token block. For such channels, even the 8-bit K error is 0.3–0.45 instead of ≈ 0.004 , and generation degenerates. In the prefill setting the code was written for, a channel spans thousands of tokens and its range almost always contains 0, which probably hid the issue. Our fix stores the minimum as the offset and uses the same bits and metadata. With the released quantizer, RDKV scores 5–13 pp below the fixed version on AIME25 for R1-Qwen and Qwen3 and collapses on R1-Llama at both budgets (0.0% vs. 31.7/35.8%, with 73/57% of samples capped). All other RDKV results use the fixed quantizer.`

Per-token and per-channel allocation did not improve accuracy here, and it used more memory. Pooled over the 18 test settings, RDKV is within noise of BreadthKV (-1.0 pp [$-2.1, +0.1$]) while holding $1.27\times$ its KV memory-time (Table 5). The per-block budget was matched on calibration-set lengths, but the cache grows with the generation, and RDKV writes 6% longer outputs. Where it is ahead, as on Qwen3 AIME25 at $B = 512$, it also uses $1.4\times$ the memory-time.

5.5 Comparison with thought-adaptive compression (ThinKV)

ThinKV (Ramachandran et al., 2026) uses attention sparsity to classify segments of the CoT as reasoning (R), execution (E) or transition (T) thoughts, and quantizes each type at its own precision. The R4E4T2 setting uses NVFP4 for R and E and ternary values for T, in groups of 16 with FP8 scales. At every transition, ThinKV anneals each earlier segment from 128 to 64, ..., 4 tokens, keeping tokens chosen by K-means on the keys. No code is available, so we re-implemented it from the paper, converting its token budget to our byte budget and keeping the prompt in full.

Reproduction choices. The paper leaves open the ternary quantizer (we use TWN), the K-means initialization (we run 8 Lloyd iterations), and how a segment’s sparsity is aggregated (we take the mean over its steps). On our models, the paper’s KDE calibration rarely finds three sparsity modes (at most 1 of 24 traces per layer), so the thresholds fall back to tertiles. Thresholds computed offline on FullKV traces also do not transfer to decoding under compression, because sparsity depends on the number of cached entries. With offline thresholds, 61–100% of runtime segments fall into a single type (at least 97% at $B = 512$) and almost none into transitions. So we set the thresholds at the tertiles of ThinKV’s own runtime sparsity on the calibration set, iterating twice.

Key format on R1-Distill-Qwen. The keys of R1-Qwen carry large per-channel offsets from the k_{proj} bias ($|K|$ up to ~ 430). With the paper’s formats (FP8 with a per-tensor scale, symmetric NVFP4 and ternary), generation collapses, and AIME25 accuracy is 0.0% at both budgets. Keeping the keys in fp16 restores normal generation. Subtracting each segment’s per-channel mean before quantization also fixes the collapse. The mean is stored in fp16 once per segment, but our byte count charges it as 0.125 bit per retained key element, which undercounts it once a segment is annealed; we bound this below. Our main ThinKV results use this centering on all models.

ThinKV scores well below BreadthKV, by 12.3 pp pooled $[-14.1, -10.5]$, and the gap is largest on Qwen3, the model with the longest traces (Table 6). The gap is not a matter of memory. As counted, ThinKV holds $0.86\times$ BreadthKV’s KV memory-time (range $0.72\text{--}1.01\times$), and at most $0.93\times$ (range $0.74\text{--}1.10\times$) if every segment’s K

Table 6: BreadthKV (BKV) vs ThinKV (with K centering): accuracy (%) averaged over each model’s three test benchmarks (as in Table 5), ThinKV’s KV memory-time relative to BreadthKV (MT; as counted, which undercounts ThinKV’s per-segment K means, see text), and the share of samples hitting the length cap (%; 32K, MATH500 16K).

model	B	accuracy			capped	
		BKV	ThinKV	MT	BKV	ThinKV
R1-Qwen	512	51.8	46.0	$0.94\times$	8	22
R1-Qwen	1024	55.0	52.2	$0.74\times$	6	12
R1-Llama	512	58.3	43.9	$0.87\times$	22	54
R1-Llama	1024	56.4	54.3	$0.91\times$	5	31
Qwen3	512	64.6	39.6	$0.86\times$	30	61
Qwen3	1024	77.1	53.6	$0.86\times$	11	42

means are charged in full. It uses less memory because annealing at each transition shrinks old segments whatever the budget. A difference of 7–14% is small next to what doubling the memory does, which changes BreadthKV’s mean accuracy by -2 to $+13$ pp per model. Where ThinKV does differ is derailment (§5.6), since its runs reach the cap far more often than BreadthKV’s. With the paper’s formats and no centering, ThinKV is 22.1 pp below BreadthKV over the 14 settings we ran and collapses on R1-Qwen.

5.6 Runs that reach the length cap

Most of what eviction loses on R1-Llama and Qwen3 comes from runs that never finish. We call a run *derailed* if it reaches the 32K cap. Derailed runs are almost never correct (at most 10 of 240–480 samples per configuration are both derailed and correct), and on these two models 74–98% of SnapKV-D’s wrong answers are derailed (Table 7; Fig. 4). On R1-Qwen, derailment accounts for a smaller share of the errors, and compression also lowers accuracy on runs that finish; on the same problems, finished SnapKV-D runs at $B = 512$ score 21.9% against 42.5% for FullKV.

A derailed run rarely loops word for word. Appendix D shows one from Qwen3-8B with SnapKV-D at $B = 512$, on an AIME24 problem that FullKV and both hybrids solve in all 8 samples. The run finds the right conditional probability within the first tenth of its output and then checks it again and again, in slightly different words each time, until it is cut off at 32K tokens without closing its reasoning.

We suspect the following mechanism, although we have not tested it directly. A long CoT keeps

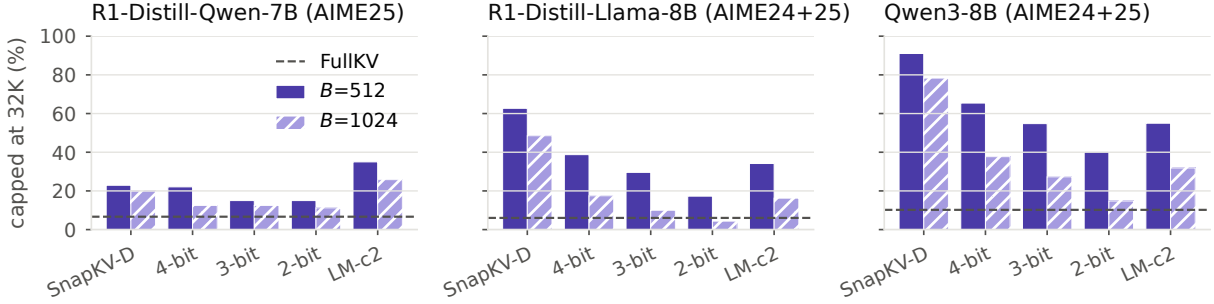


Figure 4: Share of samples hitting the 32K length cap on the test AIMEs (R1-Qwen: AIME25; others: AIME24 and AIME25), by method and budget. Dashed: FullKV.

Table 7: Derailment on the test AIMEs. acc: pass@1 (%). cap: samples hitting the 32K cap (%). c/w: share of wrong answers that are capped.

model	method	@512			@1024		
		acc	cap	c/w	acc	cap	c/w
R1-Qwen	FullKV	41.2	6.7	11%			
	SnapKV-D	20.4	22.9	29%	26.2	20.0	27%
	4-bit hyb.	29.2	22.1	31%	36.2	12.5	20%
	2-bit hyb.	35.4	15.0	23%	35.4	11.2	17%
R1-Llama	FullKV	41.2	6.0	10%			
	SnapKV-D	29.0	62.7	88%	34.4	48.5	74%
	4-bit hyb.	41.9	38.8	67%	42.5	17.5	30%
	2-bit hyb.	37.9	17.3	28%	41.5	4.4	7%
Qwen3	FullKV	69.8	10.2	32%			
	SnapKV-D	9.4	91.0	98%	18.3	78.3	94%
	4-bit hyb.	28.8	65.4	90%	57.3	37.7	86%
	2-bit hyb.	51.7	40.4	82%	69.0	14.8	47%

returning to its own intermediate results. Under eviction, a result that has not been attended to for a while is dropped, and so is the record that it was already verified. When the model later wants to rely on it, it finds nothing to build on and derives it again, and that derivation is itself evicted a few thousand tokens later. A low-bit cache keeps several times as many tokens. They are noisy, but the model can still see that the check was done and move on. The frequency of “wait” is consistent with this. On Qwen3 AIME24 at $B = 512$, SnapKV-D runs contain 6.4 “wait”s per thousand tokens, against 5.0 for the 2-bit hybrid and 3.7 for FullKV. A direct test would track whether the tokens of an earlier check are still in the cache when the model starts a new one.

At equal bytes, fewer bits per token do mean fewer derailed runs. Going from 4 to 3 to 2 bits never raises the capped share, and 2 bits is below 4 bits in all six model–budget pairs. Fewer derailed runs do not always mean higher accuracy, because fewer bits also add noise to every retained token.

On R1-Llama at $B = 512$, 4 bits derails about twice as often as 2 bits and still scores higher.

5.7 The floor of breadth: sub-2-bit and rotated codecs

To test whether breadth still helps below 2 bits, we use TurboQuant-style codecs (*LM-c*), which combine a random-sign Hadamard rotation, a per-token fp16 norm and Gaussian Lloyd–Max levels, and remove K’s per-channel mean in each 64-token group. LM-c2 costs 2.25 bits per element (KIVI-2 costs 2.5), and LM-c1 costs 1.25. Offline, LM-c makes 1 bit usable, with attention errors of 0.27–0.40 against 2.2–5.1 for uniform 1-bit quantization, and LM-c2 matches KIVI-2’s error with 10% fewer bytes.

End to end, none of them helps, and with these codecs the benefit of breadth ends at about 2 bits. LM-c1 collapses, with 0–12% calibration accuracy and 73–100% of samples capped (we stopped these runs early once this was clear), and pairing KIVI-2 keys with 1-bit LM values (1.81 bits) is much worse than KIVI-2 on Qwen3. Over the 18 test settings, LM-c2 is 2.6 pp below the KIVI-2 hybrid at equal bytes $[-3.8, -1.4]$, writes 11% longer outputs and derails more in all six AIME configurations, although its offline error is the same. Whatever makes the difference, the averaged single-step error we measure does not see it. The calibration set would have rejected LM-c2 for Qwen3, but it favored LM-c2 on the R1 models by 2.5–3.3 pp, a gain that did not hold on the test sets.

5.8 Can derailed runs be rescued after the fact?

Since derailed runs fail by never concluding, we tested on the calibration set whether forcing a conclusion helps; it mostly does not. For each capped run of the 2-bit hybrid, we cut the run at the loop

Table 8: Forced conclusion of capped runs (calibration set, 2-bit hybrid): capped runs, capped runs that become correct (recovered), and calibration accuracy (%) before $\rightarrow$ after.

model	B	capped	recovered	accuracy
R1-Qwen-7B	512	37	0	28.3 $\rightarrow$ 28.3
	1024	22	0	33.8 $\rightarrow$ 33.8
R1-Llama-8B	512	54	1	33.8 $\rightarrow$ 34.2
	1024	12	0	40.0 $\rightarrow$ 40.0
Qwen3-8B	512	115	17 (15%)	43.3 $\rightarrow$ 50.4
	1024	51	8 (16%)	62.9 $\rightarrow$ 66.2

onset found by an n -gram repetition detector (24-grams, $\geq 60\%$ repeated within a 1K window); if the detector finds no onset, we drop the last 4K tokens instead. We then append `</think>` and let the model answer under the same compressed cache (Table 8). This recovers 15% of capped runs on Qwen3 and almost none on the R1 models, below the 30% we had set in advance as the bar for running it on the test sets. Part of the problem is the detector, which almost never fires on finished runs but finds an onset in only 6 of 115 and 8 of 51 capped Qwen3 runs, because most derailed runs repeat reasoning steps and not long exact spans, as in Appendix D.

5.9 Efficiency

To see whether low-bit storage slows decoding, we implement a Triton split-K decode kernel that reads packed low-bit KV directly. Codes are stored in a planar layout, so unpacking is a shift and a mask; dot products run in fp16 on tensor cores, and the V rotation is undone once on the output. The kernel stores K as in the hybrids and V with one scale/zero pair per token instead of two (the layout of Appendix B), and its maximum relative error against a PyTorch reference is 0.7%. On an H200 (batch 64, R1-Qwen-7B heads, one layer, one decode step) at 32K context, fp16 FlashAttention takes 1.63 ms and int8/int4/int2 take 2.35/1.70/2.21 ms; at 16K, the times are 0.75 and 1.20/0.87/1.13 ms. Int4 is close to fp16 latency at 27% of the memory, so a fixed memory could hold about $3.7\times$ (int4) to $6.7\times$ (int2) more sequences. The kernel is compute-bound, because the 7 query heads per KV head are padded to 16 MMA rows, and int2 is not faster than int4.

6 Conclusion

For decode-time compression of long CoT, how a byte budget is split between tokens and precision

mattered more in our experiments than which tokens an eviction method keeps. Spending the bytes on more low-bit tokens, with the bit-width set by a short end-to-end calibration, beat eviction in 17 of 18 settings and matched a joint rate–distortion allocator that uses more memory. Offline attention error and per-head probes did not identify the bit-width, so we would not set it without an end-to-end run. The open problem is derailment on long traces. On Qwen3-8B, 40% of 2-bit runs at our tightest budget still reach the cap, and forcing a conclusion recovers few of them. If our account of derailment is right, a cache that keeps even a coarse record of what the model has already checked should help more than a better token scorer.

Limitations

Calibration. The calibration set is small and was validated at two budgets. At these budgets, always using 2 bits is statistically on par with calibration. The benefit of calibration is that it avoids the model-specific failures of other fixed choices; whether it is needed at larger budgets has not been tested on held-out data.

Statistical power. AIME has 30 problems per year, so single-setting confidence intervals are wide. Cross-setting claims rest on pooled tests. Each configuration uses one sampling seed; variance across seeds is covered only through repeated samples per problem and is not reported separately.

Models, tasks and languages. We study three open 7–8B reasoning models (two R1-distilled, one Qwen3) on English mathematics and science benchmarks; GPQA was run on one model only. We did not evaluate code generation, long-input tasks, other languages, larger models or other reasoning-model families, so the breadth–depth trade-off and the calibrated bit-widths may differ there.

Mechanism. The derailment analysis is correlational, and the mechanism in §5.6 is a hypothesis we have not tested. The rescue experiment used a literal-repetition detector, ran on calibration data only and intervened late.

Reproduced baselines. RDKV’s decode mode has no total budget, so its memory is matched only approximately, and we modified the zero point of its quantizer (we also report as-released results). ThinKV has no public code; our re-implementation fixes details the paper leaves open, calibrates thresholds on runtime sparsity and adds K-channel cen-

tering, whose per-segment means our byte count only partly charges (we report a bound). Results from the authors’ own implementations may differ.

Scope of the comparisons. The sub-2-bit study covers one codec family; vector or learned codebooks might move the floor. The low-rank baseline uses a frozen basis. We did not run every recent reasoning-specific method; R-KV represents reasoning-aware eviction.

Simulated quantization. Quantization is simulated in the engine (quantize, then dequantize) with exact byte accounting. We do not measure end-to-end memory or serving throughput; the sequence counts in §5.9 are a projection from the kernel, which uses a closely related but not identical layout.

Test contamination. Some benchmarks, MATH500 in particular, may appear in the models’ training data. All comparisons are between compression methods on the same model, so contamination should affect absolute accuracy more than the differences we report, but we cannot rule out interactions.

Ethical Considerations

This work studies inference efficiency and poses low direct risk. Compression can preserve final answers while degrading the reasoning that supports them (Ai et al., 2026), so compressed reasoning models should be evaluated beyond final-answer accuracy before deployment. We use publicly released models and benchmarks for evaluation only, consistent with their intended research use. The DeepSeek-R1-Distill models are MIT-licensed (the Llama-based one is also subject to the Llama 3.1 license), Qwen3-8B is under Apache 2.0, GPQA under CC BY 4.0 and MATH under MIT, and the AIME problems are published by the MAA; we use public copies on Hugging Face. The RDKV code is MIT-licensed. The benchmarks contain no personal data.

Use of AI assistants. We used an AI assistant (Claude) to write most of the code (the decoding engine, compression methods, baseline reproductions, kernel and analysis scripts), to run and monitor experiments, to search the literature and to draft the text. Every reported number is computed by scripts from logged generations. The author takes full responsibility for the content.

References

- Mengting Ai, Jingrui He, and Yue Guo. 2026. Does accuracy equal evidence? reasoning faithfulness under KV cache compression. *arXiv preprint arXiv:2608.01631*.
- Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. 2024. QuaRot: Outlier-free 4-Bit inference in rotated LLMs. In *Advances in Neural Information Processing Systems*. ArXiv:2404.00456.
- Priyansh Bhatnagar, Ashkan Moradifiroozabadi, Se-Hyun Yang, SeungJae Lee, Jungwook Choi, and Mingu Kang. 2026. STAR-KV: Low-rank KV cache compression via soft thresholding for adaptive rank control. *arXiv preprint arXiv:2606.08382*.
- Sayed Pedram Haeri Boroujeni, Niloufar Mehrabi, Patrick Woods, Gabriel Hillesheim, and Abolfazl Razi. 2026. Don’t waste bits! adaptive KV-Cache quantization for lightweight on-device LLMs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. ArXiv:2604.04722.
- Zefan Cai, Wen Xiao, Hanshi Sun, Cheng Luo, Yikai Zhang, Ke Wan, Yucheng Li, Yeyang Zhou, Li-Wen Chang, Jiuxiang Gu, Zhen Dong, Anima Anandkumar, Abdelkadir Asi, and Junjie Hu. 2025. R-KV: Redundancy-aware KV cache compression for reasoning models. In *Advances in Neural Information Processing Systems*. ArXiv:2505.24133.
- Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and Wen Xiao. 2024. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*.
- Chi-Chih Chang, Wei-Cheng Lin, Chien-Yu Lin, Chong-Yan Chen, Yu-Fang Hu, Pei-Shuo Wang, Ning-Chi Huang, Luis Ceze, Mohamed S. Abdelfattah, and Kai-Chiang Wu. 2025. Palu: Compressing KV-Cache with low-rank projection. In *International Conference on Learning Representations*. ArXiv:2407.21118.
- Chi-Chih Chang, Wei-Cheng Lin, Chien-Yu Lin, Hung-Yueh Chiang, Yash Akhauri, Xilai Dai, Huiqiang Jiang, Yucheng Li, Luis Ceze, Kai-Chiang Wu, and Mohamed S. Abdelfattah. 2026a. xKV: Cross-layer KV-Cache compression via aligned singular vector extraction. In *International Conference on Machine Learning*. ArXiv:2503.18893.
- Ting-Yun Chang, Harvey Yiyun Fu, Deqing Fu, Chenghao Yang, Jesse Thomason, and Robin Jia. 2026b. Value-aware stochastic KV cache eviction for reasoning models. *arXiv preprint arXiv:2606.03928*.
- Anay Chauhan, Gurucharan Marthi Krishna Kumar, Arion Das, Amit Dhanda, Vinija Jain, Aman Chadha,

- and Amitava Das. 2026. SPHERICAL KV: Angle-domain attention and rate-distortion retention for efficient long-context inference. *arXiv preprint arXiv:2605.18856*.
- Minsoo Cheong, Woosang Lim, Vincent-Daniel Yun, and Sungjoo Yoo. 2026. KV-Rescue: Recovering reasoning language model KV eviction loss via stepwise interleaving. *arXiv preprint arXiv:2608.15797*.
- Tri Dao. 2024. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations*. ArXiv:2307.08691.
- Wenjie Du, Li Jiang, Keda Tao, Xue Liu, and Huan Wang. 2026. Which heads matter for reasoning? RL-Guided KV cache compression. In *International Conference on Machine Learning*. ArXiv:2510.08525.
- Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S. Kevin Zhou. 2025. Ada-KV: Optimizing KV cache eviction by adaptive budget allocation for efficient LLM inference. In *Advances in Neural Information Processing Systems*. ArXiv:2407.11550.
- Yizhao Gao, Shuming Guo, Shijie Cao, Yuqing Xia, Yu Cheng, Lei Wang, Lingxiao Ma, Yutao Sun, Tianzhu Ye, Li Dong, Hayden Kwok-Hay So, Yu Hua, Ting Cao, Fan Yang, and Mao Yang. 2025. SeerAttention-R: Sparse attention adaptation for long reasoning. *arXiv preprint arXiv:2506.08889*.
- Ayushman Garg, Akshita Gupta, Shaswata Bhat-tacharya, Abhishek Gupta, Sandeep Kumar, and Manoj Kumar. 2026. QEvict: Recoverable quantized KV eviction for attention-drift-robust long-context decoding. *arXiv preprint arXiv:2608.05326*.
- Daya Guo, Dejian Yang, Haowei Zhang, et al. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645:633–638.
- Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems*. ArXiv:2401.18079.
- Junhao Hu, Wenrui Huang, Weidong Wang, Zhenwen Li, Tiancheng Hu, Zhixia Liu, Xusheng Chen, Tao Xie, and Yizhou Shan. 2025. RaaS: Reasoning-aware attention sparsity for efficient LLM reasoning. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 2577–2590, Vienna, Austria. Association for Computational Linguistics.
- Junhyuck Kim, Ethan Ewer, Taehong Moon, Jongho Park, and Dimitris Papailiopoulos. 2026. Not all bits are equal: Scale-dependent memory optimization strategies for reasoning models. In *International Conference on Learning Representations*. ArXiv:2510.10964.
- Steven Kolawole and Virginia Smith. 2026. Epiphany-aware KV cache eviction without the attention matrix. *arXiv preprint arXiv:2606.26472*.
- Xing Li, Zeyu Xing, Yiming Li, Linping Qu, Hui-Ling Zhen, Wulong Liu, Yiwu Yao, Sinno Jialin Pan, and Mingxuan Yuan. 2025. KVTuner: Sensitivity-aware layer-wise mixed-precision KV cache quantization for efficient and nearly lossless LLM inference. In *International Conference on Machine Learning*. ArXiv:2502.04420.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*. ArXiv:2404.14469.
- Xinyu Lian, Walid Krichene, Beichen Huang, Masahiro Tanaka, Olatunji Ruwase, Li Zhang, and Minjia Zhang. 2026. Quantization inflates reasoning: Token inflation as a hidden cost of low-bit reasoning models. *arXiv preprint arXiv:2606.25519*.
- Mengqi Liao, Lu Wang, Chaoyun Zhang, Zekai Shen, Xiaowei Mao, Si Qin, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Huaiyu Wan. 2025. G-KV: Decoding-time KV cache eviction with global attention. *arXiv preprint arXiv:2512.00504*.
- Minghui Liu, Aadi Palnitkar, Tahseen Rabbani, Hyunwoo Jae, Kyle Rui Sang, Dixi Yao, Shayan Shabihi, Fuheng Zhao, Tian Li, Ce Zhang, Furong Huang, and Kunpeng Zhang. 2025. Hold onto that thought: Assessing KV cache compression on reasoning. *arXiv preprint arXiv:2512.12008*.
- Tengxuan Liu, Shiyao Li, Jiayi Yang, Tianchen Zhao, Feng Zhou, Xiaohui Song, Guohao Dai, Shengen Yan, Huazhong Yang, and Yu Wang. 2026a. PM-KVQ: Progressive mixed-precision KV cache quantization for Long-CoT LLMs. In *International Conference on Learning Representations*. ArXiv:2505.18610.
- Wenhao Liu, Hao Shi, Yunhe Li, Weizhi Fei, Xiangyuan Wang, Mengzhe Ruan, Hanxu Hou, Peisong Wang, Linqi Song, and Shuang Qiu. 2026b. ReasonAlloc: Hierarchical decoding-time KV cache budget allocation for reasoning models. *arXiv preprint arXiv:2606.11164*.
- Xiang Liu, Zhenheng Tang, Hong Chen, Peijie Dong, Zeyu Li, Xiuze Zhou, Bo Li, Xuming Hu, and Xiaowen Chu. 2026c. Semantic integrity matters: Benchmarking and preserving high-density reasoning in KV cache compression. In *International Conference on Machine Learning*. ArXiv:2502.01941.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning*. ArXiv:2402.02750.

- Weian Mao, Xi Lin, Wei Huang, Yuxin Xie, Tianfu Fu, Bohan Zhuang, Song Han, and Yukang Chen. 2026. TriAttention: Efficient long reasoning with trigonometric KV compression. *arXiv preprint arXiv:2604.04921*.
- Lorenz K. Muller, Philippe Bich, Chiara Boretti, Hyun-Min Chang, Jiawei Zhuang, and Lukas Cavigelli. 2026. KVarN: Variance-normalized KV-Cache quantization mitigates error accumulation in reasoning tasks. *arXiv preprint arXiv:2606.03458*.
- Charilaos Pipis, Shivam Garg, Vasilis Kontonis, Vaishnavi Shrivastava, Akshay Krishnamurthy, and Dimitris Papailiopoulos. 2026. Wait, wait, wait... why do reasoning models loop? In *International Conference on Machine Learning*. ArXiv:2512.12895.
- Akshat Ramachandran, Marina Neseem, Charbel Sakr, Rangharajan Venkatesan, Brucek Khailany, and Tushar Krishna. 2026. ThinKV: Thought-adaptive KV cache compression for efficient reasoning models. In *International Conference on Learning Representations*. ArXiv:2510.01290.
- Ayan Sengupta, Siddhant Chaudhary, and Tanmoy Chakraborty. 2025. Value-guided KV compression for LLMs via approximated CUR decomposition. *arXiv preprint arXiv:2509.15038*.
- Hanzhang Shen, Haoran Wu, Yiren Zhao, and Robert Mullins. 2026. TriAxialKV: Toward extreme low-precision KV-Cache quantization for agentic inference tasks. *arXiv preprint arXiv:2605.17170*.
- Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong, Yuejie Chi, and Beidi Chen. 2025. ShadowKV: KV cache in shadows for high-throughput long-context LLM inference. In *International Conference on Machine Learning*. ArXiv:2410.21465.
- Jiayi Tian, Seyedarmin Azizi, Yequan Zhao, Erfan Baghaei Potraghloo, Sean McPherson, Sharath Nittur Sridhar, Zhengyang Wang, Zheng Zhang, Massoud Pedram, and Souvik Kundu. 2025. SkipKV: Selective skipping of KV generation and storage for efficient inference with large reasoning models. *arXiv preprint arXiv:2512.07993*.
- He Wang, Yu Gu, Fangfang Li, Zhigang Wang, Zhenghao Liu, Ning Wang, Xiaohua Li, and Ge Yu. 2026a. HqeKV: Towards hybrid quantization and eviction for KV cache in long-context LLM inference. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 4138–4153, San Diego, California, United States. Association for Computational Linguistics.
- Heng Wang, Jielin Qiu, Wenting Zhao, Cheng Qian, Liangwei Yang, Jiawei Han, Heng Ji, Silvio Savarese, Shelby Heinecke, and Huan Wang. 2026b. Random attention: Rethinking KV cache eviction for efficient reasoning. *arXiv preprint arXiv:2609.03430*.
- Haojun Xia, Xiaoxia Wu, Jisen Li, Robert Wu, Junxiong Wang, Jue Wang, Chenxi Li, Aman Singhal, Alay Dilipbhai Shah, Alpary Ariyak, Donglin Zhuang, Zhongzhu Zhou, Ben Athiwaratkun, Zhen Zheng, and Shuaiwen Leon Song. 2026. Kitty: Accurate and efficient 2-bit KV cache quantization with dynamic channel-wise precision boost. In *Proceedings of Machine Learning and Systems*. ArXiv:2511.18643.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*. ArXiv:2309.17453.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Dongjie Yang, Xiaodong Han, Yan Gao, Yao Hu, Shilin Zhang, and Hai Zhao. 2024a. PyramidInfer: Pyramid KV cache compression for high-throughput LLM inference. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 3258–3270, Bangkok, Thailand. Association for Computational Linguistics.
- June Yong Yang, Byeongwook Kim, Jeongin Bae, Beomseok Kwon, Gunho Park, Eunho Yang, Se Jung Kwon, and Dongsoo Lee. 2024b. No token left behind: Reliable KV cache compression via importance-aware mixed precision quantization. *arXiv preprint arXiv:2402.18096*.
- Jiayi Yuan, Hongyi Liu, Shaochen Zhong, Yu-Neng Chuang, Songchen Li, Guanchu Wang, Duy Le, Hongye Jin, Vipin Chaudhary, Zhaozhuo Xu, Zirui Liu, and Xia Hu. 2024. KV cache compression, but what must we give in return? a comprehensive benchmark of long context capable approaches. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 4623–4648, Miami, Florida, USA. Association for Computational Linguistics.
- Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. 2025. TurboQuant: Online vector quantization with near-optimal distortion rate. *arXiv preprint arXiv:2504.19874*.
- Hui Zeng, Daming Zhao, Pengfei Yang, WenXuan Hou, Tianyang Zheng, Hui Li, Weiye Ji, and Jidong Zhai. 2026. Lethé: Layer- and time-adaptive KV cache pruning for reasoning-intensive LLM serving. In *Proceedings of the AAAI Conference on Artificial Intelligence*. ArXiv:2511.06029.
- Haoyue Zhang, Hualei Zhang, Xiaosong Ma, Jie Zhang, and Song Guo. 2026a. LazyEviction: Lagged KV eviction with attention pattern observation for efficient long reasoning. In *Proceedings of the 64th Annual Meeting of the Association for Computational*

Linguistics (Volume 1: Long Papers), pages 36335–36352, San Diego, California, United States. Association for Computational Linguistics.

Jiebin Zhang, Dawei Zhu, Yifan Song, Wenhao Wu, Chuqiao Kuang, Xiaoguang Li, Lifeng Shang, Qun Liu, and Sujian Li. 2025. [More tokens, lower precision: Towards the optimal token-precision trade-off in KV cache compression](#). In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 8092–8105, Suzhou, China. Association for Computational Linguistics.

Junkai Zhang, Hang Guo, Luca Benini, and Yawei Li. 2026b. RDKV: Rate-distortion bit allocation for joint eviction and quantization of the KV cache. *arXiv preprint arXiv:2605.08317*.

Tao Zhang, Ziqian Zeng, Hao Peng, Huiping Zhuang, and Cen Chen. 2026c. [MixKVQ: Query-aware mixed-precision KV cache quantization for long-context reasoning](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7189–7204, San Diego, California, United States. Association for Computational Linguistics.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuan-dong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H₂O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*. ArXiv:2306.14048.

Qiyuan Zhu, Dezhi Li, Pengyu Cheng, Tianle Chen, Jiacheng Wang, Ruijie Shen, Hao Gu, Sida Lin, Zirui Liu, Jiacheng Liu, and Sirui Han. 2026. Fewer tokens, smaller cache: Reward-coordinated efficient reasoning. *arXiv preprint arXiv:2608.04771*.

Fei Zuo, Zikang Zhou, Hao Cong, Xiaoyan Xi, and Ho Fai Leung. 2026. RateQuant: Optimal mixed-precision KV cache quantization via rate-distortion theory. *arXiv preprint arXiv:2605.06675*.

A Experimental Details

Prompts. Each problem is sent as a single user turn through the model’s chat template (Qwen3 with thinking enabled), followed by “Please reason step by step, and put your final answer within `\boxed{}`.” For GPQA the suffix asks for the letter of the correct option (A–D) within `\boxed{}`; each question lists its four options as lettered choices.

Grading. We take the last `\boxed{}` after `</think>` (or anywhere in the output if the answer region has none). Math answers are compared with math-verify (0.9.0), falling back to exact string match after removing spaces; GPQA answers are

compared by option letter. Runs that hit the length cap without an answer count as wrong.

Data. MATH500 (500 problems), AIME 2024 and 2025 (30 each) and GPQA-Diamond (198) are loaded from Hugging Face copies. The calibration set is the 60 AIME 2022–2023 problems of the AIMO validation set, disjoint from all test sets.

Engine and software. Models run in bf16 in our own decoding engine (FlashAttention-2 decode, CUDA graphs, per-head variable-length caches) with PyTorch 2.8.0, Transformers 4.56.2, FlashAttention 2.8.3 and Triton 3.4.0. Generation runs used H200 and A800 GPUs and took about 344 GPU-hours in total, excluding offline analyses.

Eviction details. SnapKV-D scores decode tokens by the attention of the last 8 queries, max-pooled with kernel 7, and always keeps the 8 most recent tokens. All methods compress every 128 decode steps.

B PACT: Per-Head Online Allocation

PACT was our first attempt to choose the bit-width online, separately for each KV head. It is a negative result, and we report it here for completeness.

Format and selection. PACT uses the hybrids’ K format, with V quantized per token in a Hadamard-rotated basis ($b + 0.25$ bits); quantizing K per token gives a relative attention error of 1.92 at layer 0 at 2 bits, even after rotation. At each compression step, a head keeps its top- K_b tokens by attention from the last 32 queries, stored at b bits, where K_b is the number of tokens the budget can hold at that width.

Variants. PACT-B reads b from a budget-indexed table chosen on the dev set (512→3, 1024→4, 2048→8). PACT-LA tries each $b \in \{16, 8, 4, 3, 2\}$ that is not above the head’s current width, measures the relative change in the attention output of the last 32 queries (the probe), and picks the b with the smallest change, evaluating each b at the number of tokens it could keep once the history has grown by a factor of $\gamma = 4$. A head’s precision never increases. Without this look-ahead, the probe picks 8 bits for 88% of heads and inflates output lengths.

Offline motivation. With per-token-independent quantization on the pilot traces, letting each head choose its own bit-width lowers attention error by 20–42% relative to the best single width shared by all heads, and a probe on the last 64 queries recovers 90–95% of this

per-head oracle gain.

Dev results (R1-Qwen AIME24). With one width for all heads, the best width rises with the budget, from 2 bits at $B = 256$ (26.2%) to 3 bits at 512 (46.2%, against 40.0% at 4 bits), 4 bits at 1024 (54.2%) and 8 bits at 2048 (54.6%). At $B = 1024$, PACT-LA scores 52.5%, PACT-LA with $\gamma = 2$ 49.6%, and the probe without look-ahead 46.7% with 16.2% of samples capped, all below the fixed 4-bit width.

Test results. PACT-LA and PACT-B differ significantly in 1 of 18 test comparisons. On Qwen3-8B, PACT-B’s table (3 or 4 bits) trails the 2-bit hybrid in all six AIME and MATH500 settings (−2.0 to −18.8 pp, 5 significant), and on R1-Qwen AIME25 at $B = 2048$ its 8 bits (34.2%) trail the 4-bit hybrid (37.9%).

C Full Results

Table 9 lists every held-out test setting.

D A Derailed Run

Qwen3-8B with SnapKV-D at $B = 512$, on the AIME24 lottery problem (answer 116). FullKV and the 4- and 2-bit hybrids solve this problem in all 8 samples, using 6–16K tokens; SnapKV-D reaches the cap in 5 of 8. In the sample below, the model first states the conditional probability $1/115$ about 8% of the way into its output. The sentence “the conditional probability is $1 / 115$ ” alone then occurs 23 times before the run is cut off at 32,768 tokens without `</think>`. The excerpt starts 92% of the way into the output and is verbatim, with [...] marking omissions.

Therefore, the conditional probability is $1 / 115$. However, the problem says that the answer is m/n where m and n are coprime. 1 and 115 are coprime, so $m + n$ is $1 + 115 = 116$. But I need to check if this is correct.

Wait, but maybe I made a mistake here. [...] So yes, that seems correct.

But wait, maybe there’s a mistake in calculating the number of ways she can win a prize. Let me check again. [...] Which is $6*15 + 4*6 + 1*1 = 90 + 24 + 1 = 115$. Yes, that’s correct. So the number is 115. Therefore, the conditional probability is $1/115$. So $m + n$ is $1 + 115 = 116$.

But wait, the problem says “the probability of her winning the grand prize given that she won a prize”. So that is $1/115$? But I have a feeling that maybe I made a mistake here. Let me check with another approach. [...]

Wait, but the problem says “the probability of her winning the grand prize given that she won a prize”. So that’s $1/115$. But I’m wondering why

the answer is not something else. Maybe I made a mistake in calculating the number of ways she can win a prize? [...]

The output ends mid-sentence, while the model is computing the same count of 115 once more.

Table 9: Accuracy (%) on every held-out test setting, with the share of samples hitting the length cap (%; 32K tokens, MATH500 16K) in parentheses. RDKV: block-wise, fixed quantizer. ThinKV: with K centering; ThinKV (paper): the paper’s formats, not run on four R1-Qwen settings. FullKV does not depend on B .

model	bench.	B	FullKV	SnapKV-D	4-bit	2-bit	calibrated	RDKV	ThinKV	ThinKV (paper)
R1-Qwen	AIME25	512	41.2 (7)	20.4 (23)	29.2 (22)	35.4 (15)	33.3 (15)	28.3 (21)	23.8 (40)	0.0 (23)
R1-Qwen	AIME25	1024	41.2 (7)	26.2 (20)	36.2 (12)	35.4 (11)	36.2 (12)	38.3 (10)	30.8 (26)	0.0 (33)
R1-Qwen	MATH500	512	93.0 (4)	82.4 (12)	89.0 (8)	92.0 (5)	91.0 (4)	88.8 (5)	87.2 (10)	–
R1-Qwen	MATH500	1024	93.0 (4)	86.8 (8)	92.6 (4)	91.2 (4)	92.6 (4)	91.4 (5)	91.0 (5)	–
R1-Qwen	GPQA	512	36.4 (5)	23.7 (6)	29.7 (6)	34.0 (3)	31.1 (6)	30.8 (5)	27.1 (15)	–
R1-Qwen	GPQA	1024	36.4 (5)	29.4 (6)	36.2 (3)	36.9 (1)	36.2 (3)	35.7 (2)	34.7 (4)	–
R1-Llama	AIME24	512	51.2 (6)	31.7 (60)	53.3 (32)	45.8 (15)	52.5 (27)	45.4 (19)	27.5 (74)	26.2 (65)
R1-Llama	AIME24	1024	51.2 (6)	41.7 (46)	50.4 (15)	50.0 (3)	50.0 (3)	48.3 (4)	45.0 (40)	37.5 (28)
R1-Llama	AIME25	512	31.2 (6)	26.2 (65)	30.4 (45)	30.0 (19)	32.9 (32)	31.7 (25)	21.2 (72)	21.7 (68)
R1-Llama	AIME25	1024	31.2 (6)	27.1 (51)	34.6 (20)	32.9 (5)	32.9 (5)	35.8 (5)	29.2 (45)	28.7 (38)
R1-Llama	MATH500	512	89.4 (5)	82.8 (11)	88.2 (8)	88.0 (6)	89.6 (6)	86.4 (7)	82.8 (15)	84.0 (12)
R1-Llama	MATH500	1024	89.4 (5)	87.0 (8)	89.6 (5)	86.2 (6)	86.2 (6)	89.4 (5)	88.8 (6)	86.6 (7)
Qwen3	AIME24	512	74.2 (6)	12.9 (88)	35.8 (58)	62.1 (30)	62.1 (30)	61.3 (33)	21.7 (78)	16.2 (87)
Qwen3	AIME24	1024	74.2 (6)	25.0 (74)	64.2 (30)	75.0 (10)	75.0 (10)	72.9 (16)	43.8 (50)	32.5 (70)
Qwen3	AIME25	512	65.4 (14)	5.8 (94)	21.7 (73)	41.2 (51)	41.2 (51)	47.9 (50)	14.6 (86)	11.7 (90)
Qwen3	AIME25	1024	65.4 (14)	11.7 (82)	50.4 (45)	62.9 (19)	62.9 (19)	60.8 (25)	27.5 (65)	20.0 (78)
Qwen3	MATH500	512	94.8 (5)	73.0 (28)	84.8 (15)	90.4 (9)	90.4 (9)	86.4 (14)	82.6 (17)	77.2 (23)
Qwen3	MATH500	1024	94.8 (5)	81.8 (17)	92.2 (7)	93.4 (5)	93.4 (5)	91.6 (7)	89.6 (10)	87.2 (13)